AI ACADEMY · LEVEL 5 · GRADE 10 · WEEK 26 · STUDENT LESSON

Embeddings and neighbourhoods

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Image-pattern Network Investigation

THE CONNECTED WEEK

Look up the row. Keep the mask. Measure the candidates. Retain the tie.

Trace embedding representations and geometric neighbours without inventing semantic evidence.

Keep one habit: Complete table/sequence lookup → all-candidate distance table and graph → separate query movement → isolated row fitting → executable audit → independent new table.

MAIN LESSON · SECTION 1

Which row does the ID select,and which candidate is actually nearest?

> Keep table version,mask,distance and candidate policy with every result.

Learning goals

I can look up a vector using a token ID and preserve sequence shape.

I can calculate nearest neighbours with an explicit distance and tie rule.

I can distinguish constructed vectors from learned features and limit similarity claims.

Retrieve sequence IDs and real-position masks

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Sequence representations. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from **Training** a small image model. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Image tensors and channels. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Prerequisite and starting response

- Why should numerical differences between token IDs not be treated as differences in meaning?

> **My starting explanation**

End goal

By the end, I can explain embeddings and **neighbourhood**s using a traced example and a stated limitation.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

This week you will practise:

Look up a vector using a token ID and preserve sequence shape

Calculate nearest neighbours with an explicit distance and tie rule

Distinguish constructed vectors from learned features and limit similarity claims

Retrieve what you already know

This week's end goal

MAIN LESSON · SECTION 2

Look up a vector without changing its sequence position

> Six rows and two coordinates per row describe the table;four positions describe the example sequence.

Constructed original table and vocabulary mapping

Continue the fictional robot vocabulary and add WAIT as ID 5. Use the table below, chosen by the lesson author for arithmetic practice. These coordinates are not measured robot properties and were not learned from data.

Each token ID selects one row; the resulting two-component vector is its embedding in this example.

Symbol	ID	Vector
PAD	0	[0, 0]
UNK	1	[-1, -1]

Symbol	ID	Vector
GO	2	[2, 0]
TURN	3	[1, 1]
STOP	4	[0, 2]
WAIT	5	[0, 1]

Complete four-position lookup

GO STOP with two padding slots	Token ID	Looked-up vector	Mask
Position0:GO	2	[2, 0]	1
Position1:STOP	4	[0, 2]	1
Position2:PAD	0	[0, 0]	0
Position3:PAD	0	[0, 0]	0

The table shape is 6 by 2: six vocabulary entries and **embedding dimension** 2. The ID sequence [2, 4, 0, 0] becomes [[2, 0], [0, 2], [0, 0], [0, 0]], with shape 4 by 2. Its mask remains [1, 1, 0, 0].

The vector dimension is not the sequence length or vocabulary size.

Keep the real-position mask in a vector average

A zero PAD vector does not remove the need for masking. Averaging all four vectors above gives [0.5, 0.5]; averaging only the two real vectors gives [1, 1].

UNK is a real position when it replaces an unknown symbol, so its vector is not automatically excluded.

Name given and missing facts

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Calculate every matching-coordinate difference and retain all ties

> Nearest means nearest under this table,metric and candidate set.

Complete squared-distance definition

Use **squared Euclidean distance**: square each component difference and add. From GO [2, 0] to TURN [1, 1], the distance is (2 - 1) squared + (0 - 1) squared = 2. To STOP it is 8; to WAIT it is 5.

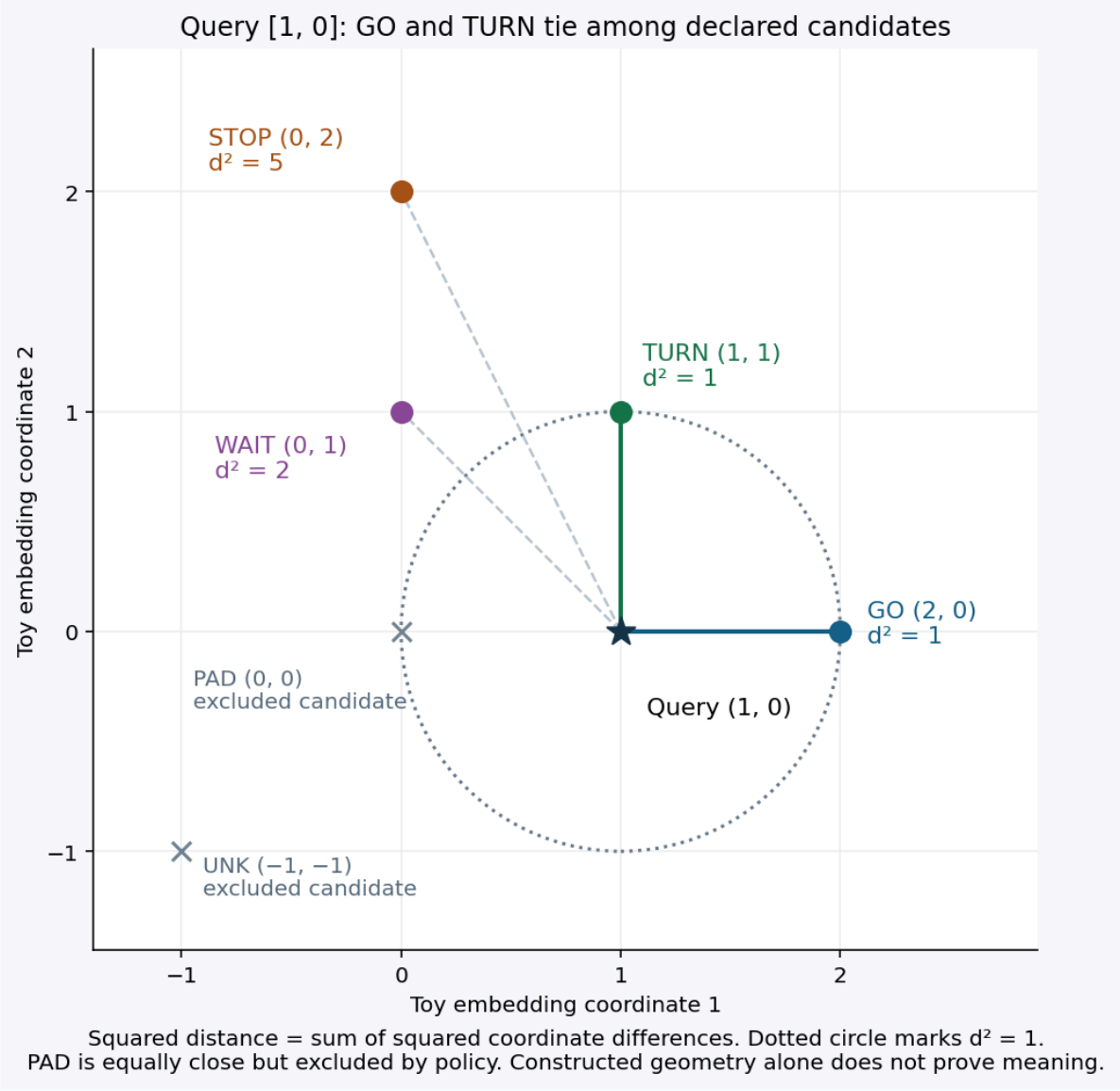
TURN is therefore the nearest of those three under this table and metric.

Both complete query/candidate comparisons

Candidate in original table	Vector	Squared distance from GO[2, 0]	Squared distance from query[1, 0]
GO	[2, 0]	Excluded self	1
TURN	[1, 1]	2	1
STOP	[0, 2]	8	5
WAIT	[0, 1]	5	2

Candidate in original table	Vector	Squared distance from GO[2, 0]	Squared distance from query[1, 0]
PAD /UNK	[0, 0]/[−1, −1]	Excluded reserved roles	Excluded reserved roles

Labelled original-table tie and excluded entries



Candidate,precision and finite-input contract

For a named-token query GO, exclude GO itself as well as reserved PAD/UNKroles. For the arbitrary query[1, 0], use all four regular candidates; it is not a named token in the table. nearest returns every exact nearest tie, sorted only for display. These integer coordinates make exact ties checkable; other numerical experiments need a declared precision policy.

The functions check nonempty/equal dimensions and nonempty candidates but assume finite numeric components. All coordinates in this original table were hand-chosen.

The candidate set excludes PAD, UNK and the query token itself. Otherwise a query would normally retrieve itself at distance zero.

This code assumes finite numeric components and uses exact ties, appropriate for these integer examples. Floating-point comparisons in other experiments need a declared precision policy.

Runnable geometry with visible results

```
table = [[0, 0], [-1, -1], [2, 0],
         [1, 1], [0, 2], [0, 1]]
known = {"GO": 2, "TURN": 3, "STOP": 4, "WAIT": 5}

def distance2(a, b):
    if len(a) != len(b) or not a:
        raise ValueError("Need equal nonempty dimensions")
    return sum((x - y)**2 for x, y in zip(a, b))

def nearest(query, candidates):
    if not candidates:
        raise ValueError("Need candidates")
    scores = {name: distance2(query, vector)
              for name, vector in candidates.items()}
    best = min(scores.values())
    return sorted(name for name, d in scores.items() if d == best)

candidates = {name: table[i] for name, i in known.items()
              if name != "GO"}
assert nearest(table[2], candidates) == ["TURN"]
print('GO query nearest among TURN/STOP/WAIT',nearest(table[2],candidates))
regular={name:table[i] for name,i in known.items()}
print('query [1,0] squared distances',{name:distance2([1,0],v) for name,v in
regular.items()})
print('query [1,0] all nearest ties',nearest([1,0],regular))
```

Connect each diagram value to the sum of squares

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

<details>
<summary>Reference: original assertion-only neighbour code</summary>

```
table = [[0, 0], [-1, -1], [2, 0],
         [1, 1], [0, 2], [0, 1]]
known = {"GO": 2, "TURN": 3, "STOP": 4, "WAIT": 5}

def distance2(a, b):
    if len(a) != len(b) or not a:
        raise ValueError("Need equal nonempty dimensions")
    return sum((x - y)**2 for x, y in zip(a, b))

def nearest(query, candidates):
    if not candidates:
        raise ValueError("Need candidates")
    scores = {name: distance2(query, vector)
              for name, vector in candidates.items()}
    best = min(scores.values())
    return sorted(name for name, d in scores.items() if d == best)

candidates = {name: table[i] for name, i in known.items()
              if name != "GO"}
assert nearest(table[2], candidates) == ["TURN"]
```

</details>

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Connect the picture and the explanation

</details>

MAIN LESSON · SECTION 4

Change the query while keeping the table fixed

> A new geometric neighbour is not evidence that a word changed meaning.

Current distance/tie task

- For query [1, 0], compute squared distances to GO, TURN, STOP and WAIT in the original table. Return all nearest candidates.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the pictured query move

- Change: Move the query to [2, 1].

▼ **Check after predicting**

Prediction: Q becomes nearest with squared distance 1; P distance 4; R distance 10.

Reason: Changing the query position changes geometric neighbours while the stored table remains fixed.

▼ **Earlier wording, alternatives and picture transcripts**

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

See how a query move changes a geometric neighbourhood

> Name the table and query before comparing the distances.

Complete separate pictured vectors and scope

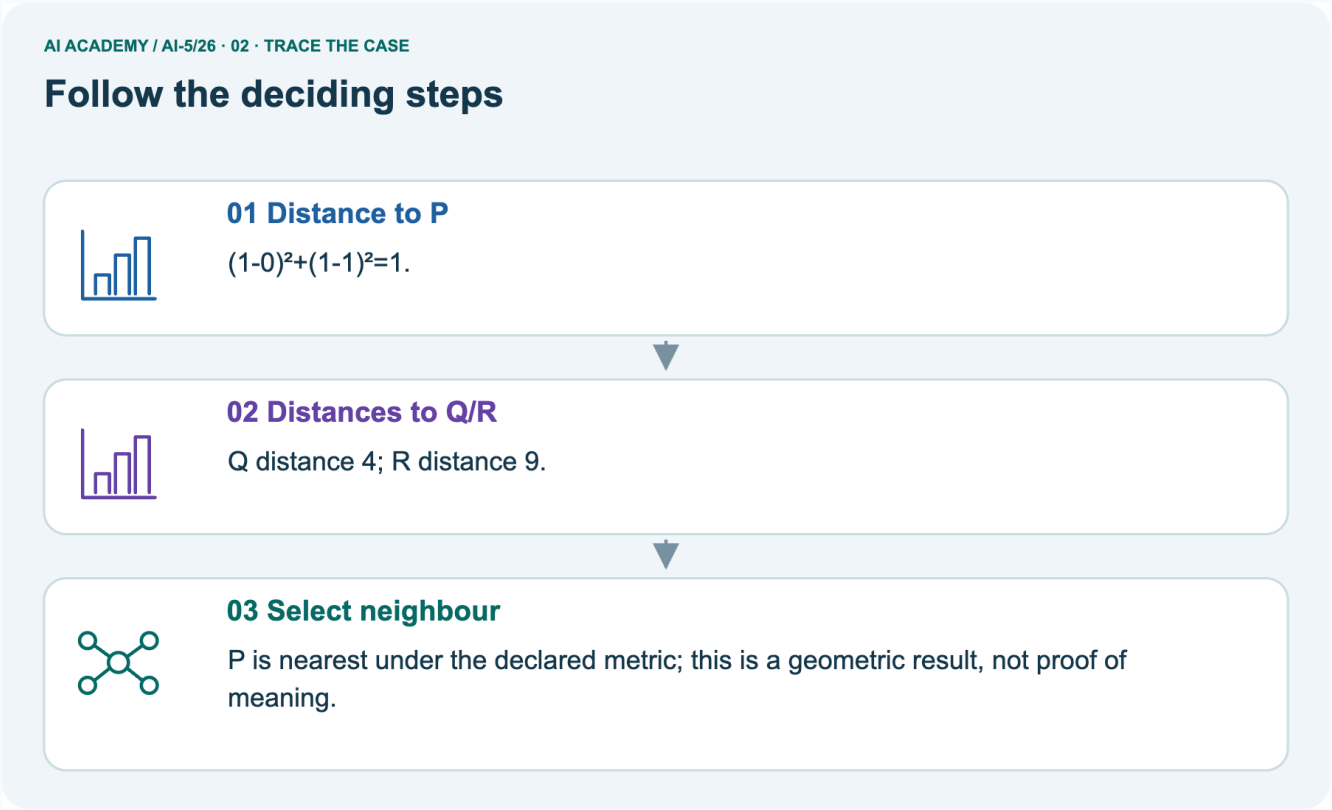
Measure embedding neighbours without turning numerical proximity into factual certainty.

Toy table P =[0, 1], Q=[3, 1], R=[1, 4]; query [1, 1]. Use **squared Euclidean distance**.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

The picture is a separate three-vector table:P[0, 1], Q[3, 1], R[1, 4]. Query[1, 1]has squared distances 1, 4, 9, so P is nearest. Move only the query to[2, 1]:distances 4, 1, 10, so Q is nearest. The original GO/TURN/STOP/WAITtable is unchanged and uses different coordinates/names. Both examples establish declared geometry, not factual truth, topic identity or a useful substitution rule.

Trace all original query distances



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Distance to P	$(1-0)^2+(1-1)^2=1$.
2. Distances to Q/R	Q distance 4; R distance 9.
3. Select neighbour	P is nearest under the declared metric; this is a geometric result, not proof of meaning.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

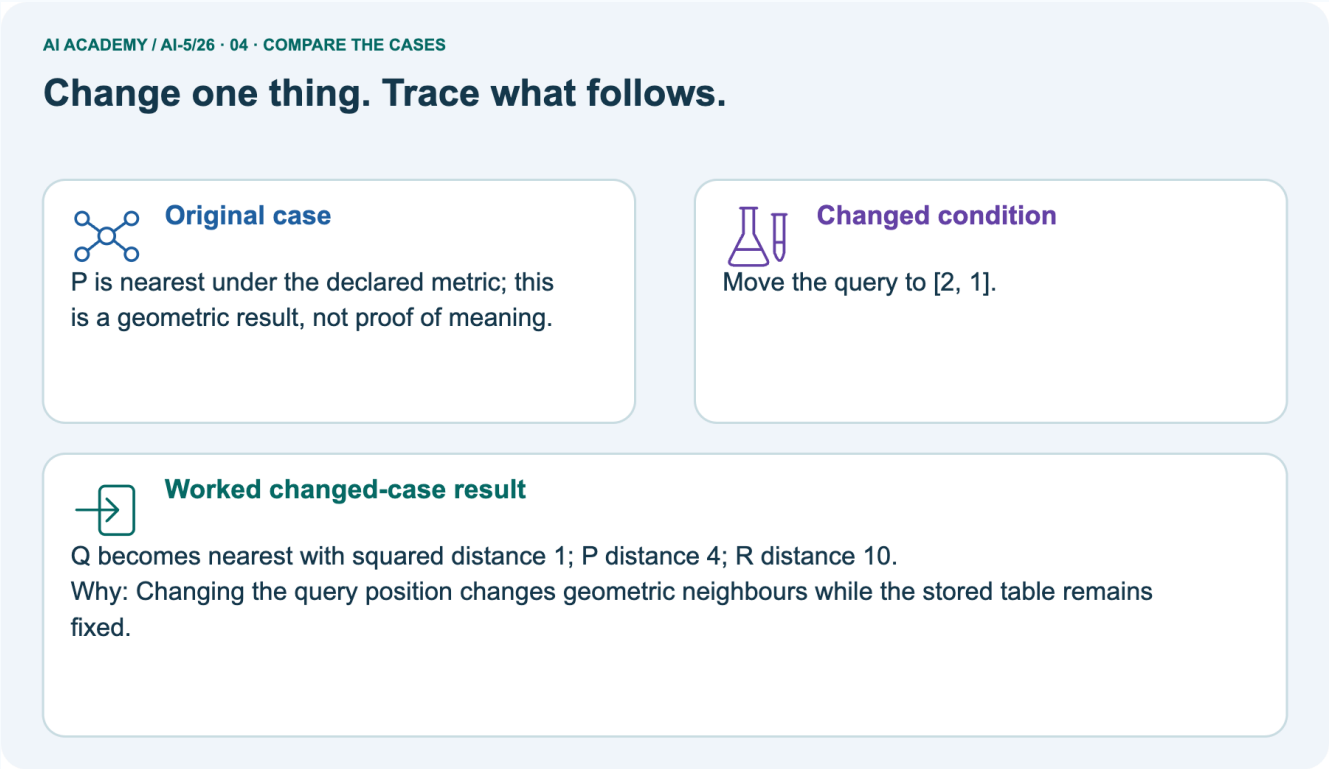
An embedding stores each item as an ordered vector. Under **squared Euclidean distance**, subtract matching coordinates, square their differences and add. The smallest total identifies the nearest supplied vector.

These invented coordinates let us check geometry exactly; a claim that neighbours share meaning needs **evidence** about how the representation was learned and used.

Predict the query move

- Change one thing: Move the query to [2, 1]. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain evidence needed beyond invented geometry

- Embedding geometry depends on **training**, metric and scale; proximity does not prove truth, causation or source permission.

- Read the labelled picture: What would justify claiming nearby vectors mean related topics? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Try STOP as a query in the original table

One query can have two closest candidates

CANDIDATE VECTOR	QUERY VECTOR	SQUARED DISTANCE
GO [2, 0]	[1, 0]	1
TURN [1, 1]	[1, 0]	1
WAIT [0, 1]	[1, 0]	2
STOP [0, 2]	[1, 0]	5

The nearest set is GO and TURN. Alphabetical display order does not break the mathematical tie or make one candidate more similar.

Using STOP [0, 2] as the query, calculate distances to GO, TURN and WAIT. Exclude STOP itself and both reserved entries. Which candidate is nearest?

▼ Earlier wording, alternatives and picture transcripts

Learn from the worked case

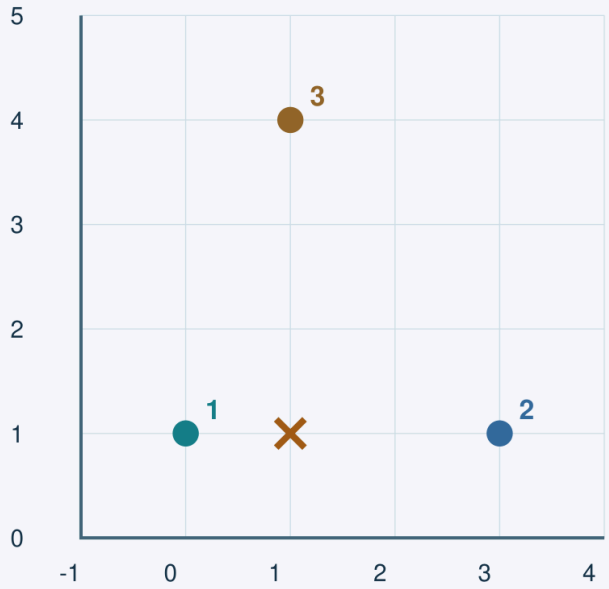
Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Embeddings and neighbourhoods

The facts we will use

AI ACADEMY / AI-5/26

Squared Euclidean embedding distances



Horizontal: first embedding coordinate; vertical: second embedding coordinate

1: P at (0, 1)

2: Q at (3, 1)

3: R at (1, 4)

Cross: query at (1, 1)

Distance rule: Squared Euclidean distance

Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / AI-5/26

Embeddings and neighbourhoods

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Distance to P

(1-0)2+(1-1)2=1.



2. Distances to Q/R

Q distance 4; R distance 9.



3. Select neighbour

P is nearest under the declared metric; this is a geometric result, not proof of meaning.

CHANGE ONE THING

Move the query to [2, 1].

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Separate an adjustable table row from a justified meaning claim

> Keep the original lookup table and isolated updated copy as different versions.

Complete isolated objective and its limited conclusion

An **embedding table** can be trainable: an optimisation procedure adjusts its entries to reduce a declared objective.

To isolate one update, let the GO row e start at [2, 0] and use the classroom loss J(e) = (e1 - 0) squared + (e2 - 2) squared.

The target vector [0, 2] is chosen for this arithmetic demonstration, not discovered **evidence** about the word GO.

The gradient at the start is [4, -4]. With learning rate 0.25, subtract the step to obtain e = [1, 1]. Loss decreases from 8 to 2. Keep every other table row frozen in this isolated example.

GO now has the same vector as TURN, so a downstream rule receiving only this vector cannot distinguish those two tokens.

This update proves only that the declared objective decreased. It does not prove that the new **neighbourhood** reflects useful meaning. Learned **neighbourhood**s depend on the **training** data, objective and model.

A nearby token need not be a safe substitute, have the same label, or be interchangeable in a sentence or command.

Every gradient,updated value and collision

Separate isolated GO-row fitting example	Complete value / status
Original GO row and chosen target	e[2, 0], target[0, 2]; target author-supplied for arithmetic
Artificial objective	$J(e)=(e1-0)^2+(e2-2)^2$
Gradient and rate	[4, −4], rate.25
New row in a copied table	[1, 1]; every other row remains frozen

Separate isolated GO-row fitting example	Complete value / status
Objective before /after	8→2
Collision with TURN	Both rows now[1, 1]; a vector-only deterministic head cannot distinguish them
Evidence limit	One defined objective decreased; useful semantic neighbourhoods were not established

Try the GO-row update

- For the isolated GO-row update, give the gradient, updated vector, starting loss and new loss. Which other token now shares the vector?

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise lookup,geometry and interpretation with less help

> Retain candidate names and the original-versus-updated table version.

Current task and response

- Using the original table, embed TURN WAIT STOP with one right-padding slot. Give the vector sequence, mask and shape.

Using STOP [0, 2] as the query, calculate distances to GO, TURN and WAIT. Exclude STOP itself and both reserved entries. Which candidate is nearest?

Explain your prediction to a partner before comparing results.

Supported W2 record

Use workbook W2 for the connected example “Embeddings and **neighbourhood**s”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Individual misconception check

- Use the worked case for Embeddings and **neighbourhood**s. Proposed explanation: “Token ID differences define embedding distances”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Pictured proximity-claim reasoning

- What would justify claiming nearby vectors mean related topics? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

> **My explanation using a pictured step and the given facts**
>
> _____
>
> _____
>
> _____
>
> _____

Reserve the independent new candidate table

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Calculate distances and a tie

Check your reasoning

Explain the picture

Run a complete lookup,mask and neighbourhood audit

> All token rows,sequence IDs,queries and fitting targets are supplied.

Current complete distance/tie task

For query [1, 0], compute squared distances to GO, TURN, STOP and WAIT in the original table. Return all nearest candidates.

Complete runnable embedding audit

```
table = [[0, 0], [-1, -1], [2, 0],
         [1, 1], [0, 2], [0, 1]]
known = {"GO": 2, "TURN": 3, "STOP": 4, "WAIT": 5}

def distance2(a, b):
    if len(a) != len(b) or not a:
        raise ValueError("Need equal nonempty dimensions")
    return sum((x - y)**2 for x, y in zip(a, b))

def nearest(query, candidates):
    if not candidates:
        raise ValueError("Need candidates")
    scores = {name: distance2(query, vector)
              for name, vector in candidates.items()}
    best = min(scores.values())
    return sorted(name for name, d in scores.items() if d == best)

candidates = {name: table[i] for name, i in known.items()
              if name != "GO"}
assert nearest(table[2], candidates) == ["TURN"]
# Complete lookup, masking and isolated row-update probes.
def lookup(ids):
    if any(type(i) is not int or not 0 <= i < len(table) for i in ids):
        raise ValueError('Token ID is outside this saved vocabulary')
    return [table[i][:] for i in ids]

def masked_vector_mean(vectors,mask):
    if not vectors or len(vectors)!=len(mask) or any(m not in (0,1) for m in mask):
        raise ValueError('Need aligned vectors and binary mask')
    width=len(vectors[0])
    if width==0 or any(len(v)!=width for v in vectors):
        raise ValueError('Need consistent nonempty vector dimensions')
```

```
    count=sum(mask)
    if count==0:
        raise ValueError('No real vectors')
    return [sum(v[j] for v,m in zip(vectors,mask) if m)/count for j in range(width)]

ids,mask=[2,4,0,0],[1,1,0,0]
vectors=lookup(ids)
print('table shape',len(table),len(table[0]),'sequence
shape',len(vectors),len(vectors[0]))
print('looked-up sequence',vectors,'mask',mask)
print('masked mean',masked_vector_mean(vectors,mask))
print('incorrect all-slot mean',masked_vector_mean(vectors,[1,1,1,1]))
print('real UNK retained',masked_vector_mean(lookup([1,4,0,0]),mask))
regular={name:table[i] for name,i in known.items()}
query=[1,0]
print('query [1,0] squared distances',{name:distance2(query,v) for name,v in
regular.items()})
print('all nearest ties',nearest(query,regular))
print('GO STOP / STOP GO same masked mean',masked_vector_mean(lookup([2,4]),
[1,1]),masked_vector_mean(lookup([4,2]),[1,1]))
old=table[2][:]
target=[0,2]
gradient=[2*(e-t) for e,t in zip(old,target)]
new=[e-.25*g for e,g in zip(old,gradient)]
updated=[row[:] for row in table]
updated[2]=new
print('isolated GO-row gradient / new vector',gradient,new)
print('artificial objective before/after',distance2(old,target),distance2(new,target))
print('GO now collides with frozen TURN',updated[2]==updated[3])
assert table[2]==[2,0]  # Original saved table remains unchanged.
# Finite numeric components assumed. No semantic training dataset or quality result is
supplied.
```

Normal,mask,ID-range,empty and collision probes

Complete embedding probe	Expected result / action
Normal table/sequence lookup	Table6×2, GO STOPplusPAD/PADgives sequence 4×2 and mask 1100
Masked versus all-slot mean	Real GO/STOPmean[1, 1]; all four slots[.5, .5]
Real UNKplusSTOP	Mask keeps UNK:mean[-.5, .5], not only STOP[0, 2]

Complete embedding probe	Expected result / action
Query[1, 0]with regular candidates	Distances:GO=1, TURN=1, STOP=5, WAIT=2; return both GO and TURN
Reserved PAD[0, 0]also lies at distance 1	Its role excludes it from this candidate set; numeric coordinates alone do not define the role
Empty candidate set or mismatched distance dimensions	Original functions raise ValueError
Negative/out-of-range token ID	Extended lookup raises ValueError before Python indexing can select a wrong row
All-zero vector mask or inconsistent vector dimensions	Extended mean rejects the undefined/malformed calculation
GO STOPversusSTOP GO	Same masked mean[1, 1], also equals single TURN; mean alone loses order and can lose length
Isolated GO row update	Gradient[4, −4], new[1, 1], J8→2; original table stays unchanged

Predict,look up,measure and bound the claim

1. Predict the lookup shapes, all candidate distances, masked means and isolated update before running or tracing the complete audit. It supplies every row and checks ID range, mask alignment and vector dimensions in the extended helpers. Record original and copied updated tables separately. Masked means use vectors at real positions; they do not average nominal token IDs. Keep all nearest ties and the candidate policy.

   A lower artificial row objective or a nearby vector does not prove useful language meaning, interchangeability or independent task performance.

2. On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Keep the evidence ledger

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Reference: original compact neighbour functions

```
table = [[0, 0], [-1, -1], [2, 0],
         [1, 1], [0, 2], [0, 1]]
known = {"GO": 2, "TURN": 3, "STOP": 4, "WAIT": 5}

def distance2(a, b):
    if len(a) != len(b) or not a:
        raise ValueError("Need equal nonempty dimensions")
    return sum((x - y)**2 for x, y in zip(a, b))

def nearest(query, candidates):
    if not candidates:
        raise ValueError("Need candidates")
    scores = {name: distance2(query, vector)
              for name, vector in candidates.items()}
    best = min(scores.values())
    return sorted(name for name, d in scores.items() if d == best)

candidates = {name: table[i] for name, i in known.items()
              if name != "GO"}
assert nearest(table[2], candidates) == ["TURN"]
```

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: use a separate candidate table and its own names

> Apply its squared-distance rule and keep every nearest tie.

Complete independent vectors and mean task

A separate table contains A [0, 0], B [2, 0] and C [1, 2]. For query [1, 0], give all squared distances and every nearest candidate. Then calculate the mean of A and B and compare it with the query.

Can that mean identify which ordered pair produced it?

First attempt and support record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback after trying

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Find a neighbourhood independently

Attempt this without hints

A separate table contains A [0, 0], B [2, 0] and C [1, 2]. For query [1, 0], give all squared distances and every nearest candidate. Then calculate the mean of A and B and compare it with the query.

Can that mean identify which ordered pair produced it?

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

MAIN LESSON · SECTION 11

Save the vocabulary and table as one versioned contract

> The project record should identify masks,metrics,candidates and any fitted row changes.

Project artifact and evidence table

Add a labelled embeddings and **neighbourhood**s **evidence** sheet to Image-pattern Network Investigation. Show how you can look up a vector using a token id and preserve sequence shape.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Actual run/trace and remaining limits

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Finish with a geometric result and its evidence boundary

> Explain what lookup,averaging and one row update each establish.

Retrieve and explain

- Close your notes. Explain this goal in your own words: Look up a vector using a token ID and preserve sequence shape

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Use the mathematical words precisely**

REFERENCE · SECTION 5

Reference · Use the mathematical words precisely

> Connect each term to a value or step in the current example.

Vocabulary and your explanation

Word	Meaning
Embedding table	A table assigning a fixed-length numerical vector to each token ID
Embedding dimension	The number of components in each embedding vector
Neighbourhood	Nearby candidates under a stated representation and distance rule
Squared Euclidean distance	The sum of squared component differences between two vectors

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain a stronger case

REFERENCE · SECTION 10

Extra challenge · Explain a stronger case

> Choose the next challenge after the main route.

Explain,repair and extend

- Using the original table, embed TURN WAIT STOP with one right-padding slot. Give the vector sequence, mask and shape.
- For the isolated GO-row update, give the gradient, updated vector, starting loss and new loss. Which other token now shares the vector?
- A developer swaps the IDs of GO and STOP but leaves the embedding rows unchanged. What goes wrong, and what must be saved or changed together?

Explain → apply → challenge

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Reference · Check the representation and its limits

REFERENCE · SECTION 12

Reference · Check the representation and its limits

> Use after practice;reserve the new independent A/B/Ccandidate and mean comparison for the first individual attempt.

Current explanation and response

- In the original table, the masked average for GO STOP is [1, 1], and STOP GO has the same average. It also equals the single TURN vector.

  Averaging discarded order and can collide across different sequence lengths unless additional information is retained. Keeping the sequence of vectors makes order available; a later model must still use it.

- Save the vocabulary-to-row mapping, table version, mask convention, distance rule and candidate policy with the model. Compare embedding choices using **training** and validation discipline, and keep final evaluation reserved.

  Next week examines how normalisation and similarity measures can change neighbour rankings.

- A developer swaps the IDs of GO and STOP but leaves the embedding rows unchanged. What goes wrong, and what must be saved or changed together?

> **A teammate says TURN must mean the same as GO because it is the closest candidate, and says a lower embedding-training loss proves this. Explain what the calculations establish and what evidence is missing.**

My limited claim and a useful next check

▼ **Extra practice · Revisit the complete trace**

REFERENCE · SECTION 13

Extra practice · Revisit the complete trace

> Predict before reading the supplied steps.

Complete trace and evidence

Complete evidence record

One query can have two closest candidates

CANDIDATE VECTOR	QUERY VECTOR	SQUARED DISTANCE
GO [2, 0]	[1, 0]	1
TURN [1, 1]	[1, 0]	1
WAIT [0, 1]	[1, 0]	2
STOP [0, 2]	[1, 0]	5

The nearest set is GO and TURN. Alphabetical display order does not break the mathematical tie or make one candidate more similar.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Reference · The earlier route checklist

REFERENCE · SECTION 15

Reference · The earlier route checklist

> Use one main route rather than repeating the lesson.

Targets,route and first prediction

Start with this question: Measure embedding neighbours without turning numerical proximity into factual certainty.

By the end, I can	How I will show it
Look up a vector using a token ID and preserve sequence shape	Explain the deciding step, show a trace or test, and state one limit.
Calculate nearest neighbours with an explicit distance and tie rule	Explain the deciding step, show a trace or test, and state one limit.
Distinguish constructed vectors from learned features and limit similarity claims	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Extra practice · Predict,check and repair

REFERENCE · SECTION 17

Extra practice · Predict,check and repair

> Keep supported feedback separate from the independent case.

First supported question and optional hint

- Measure embedding neighbours without turning numerical proximity into factual certainty.
- Which labelled step matches this detail? P is nearest under the declared metric; this is a geometric result, not proof of meaning.
- 1. Distance to P
- 2. Select neighbour
- 3. Distances to Q/R
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check after the first attempt**

After your attempt, compare with the supported explanation: Select neighbour P is nearest under the declared metric; this is a geometric result, not proof of meaning. An embedding stores each item as an ordered vector.

Under **squared Euclidean distance**, subtract matching coordinates, square their differences and add. The smallest total identifies the nearest supplied vector.

These invented coordinates let us check geometry exactly; a claim that neighbours share meaning needs **evidence** about how the representation was learned and used.

Changed-case question and optional hint

- Changed-case practice: Move the query to [2, 1]. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. This one example guarantees correct results for every future **input**.
- 3. Q becomes nearest with squared distance 1; P distance 4; R distance 10.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check after the changed attempt**

After your attempt, compare with the supported explanation: Q becomes nearest with squared distance 1; P distance 4; R distance 10. Changing the query position changes geometric neighbours while the stored table remains fixed.

Repair and retain the record

Review Sequence representations (ai-5/25). Goal: Encode an ordered sequence with an explicit vocabulary. Short practice: Which record lets the program distinguish the first and last zeros?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review **Training** a small image model (ai-5/23). Goal: Trace a complete image model with fixed features and one trained weight. Short practice: Does lower **training** loss alone establish that four updates generalise better?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review Image tensors and channels (ai-5/20). Goal: Read a pixel using declared row column and channel axes. Short practice: Where is the bottom green pixel in the first batched image?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

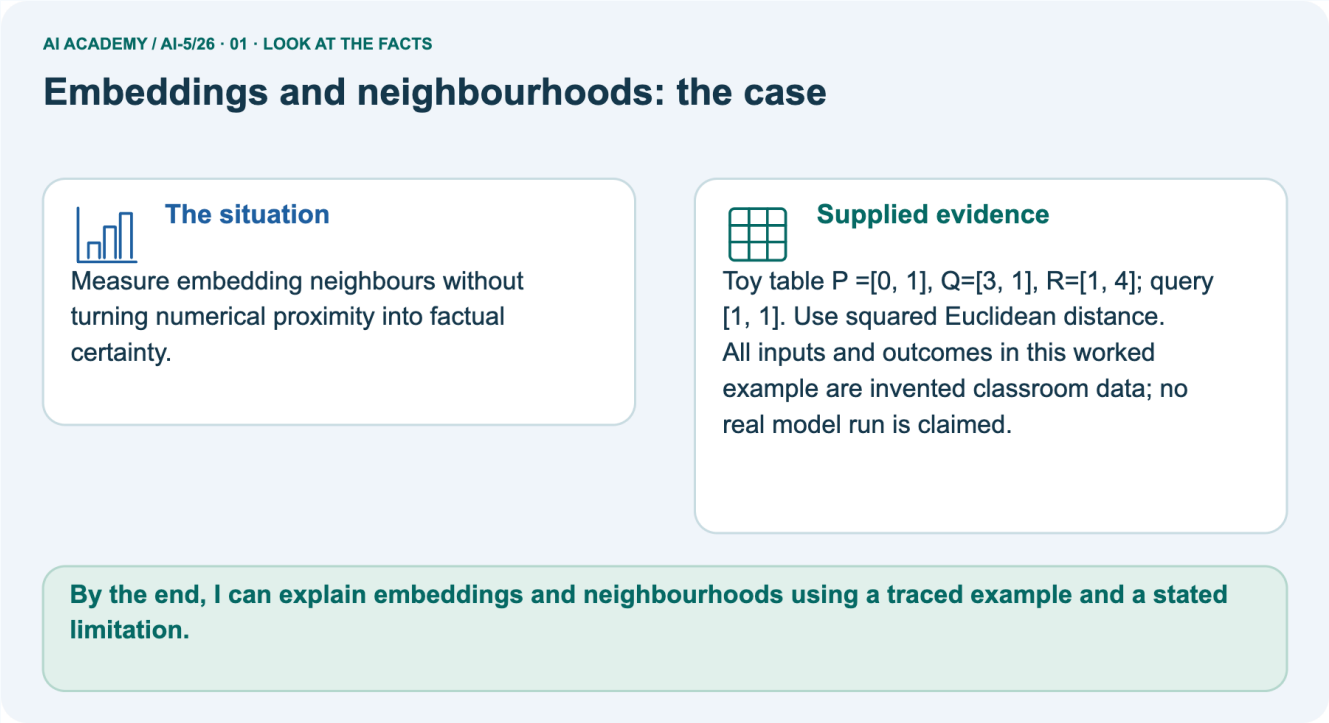
▼ Reference · Six-picture concept journey

REFERENCE · SECTION 18

Reference · Six-picture concept journey

> Choose the picture that supports the current explanation.

01. Embeddings and neighbourhoods: the case



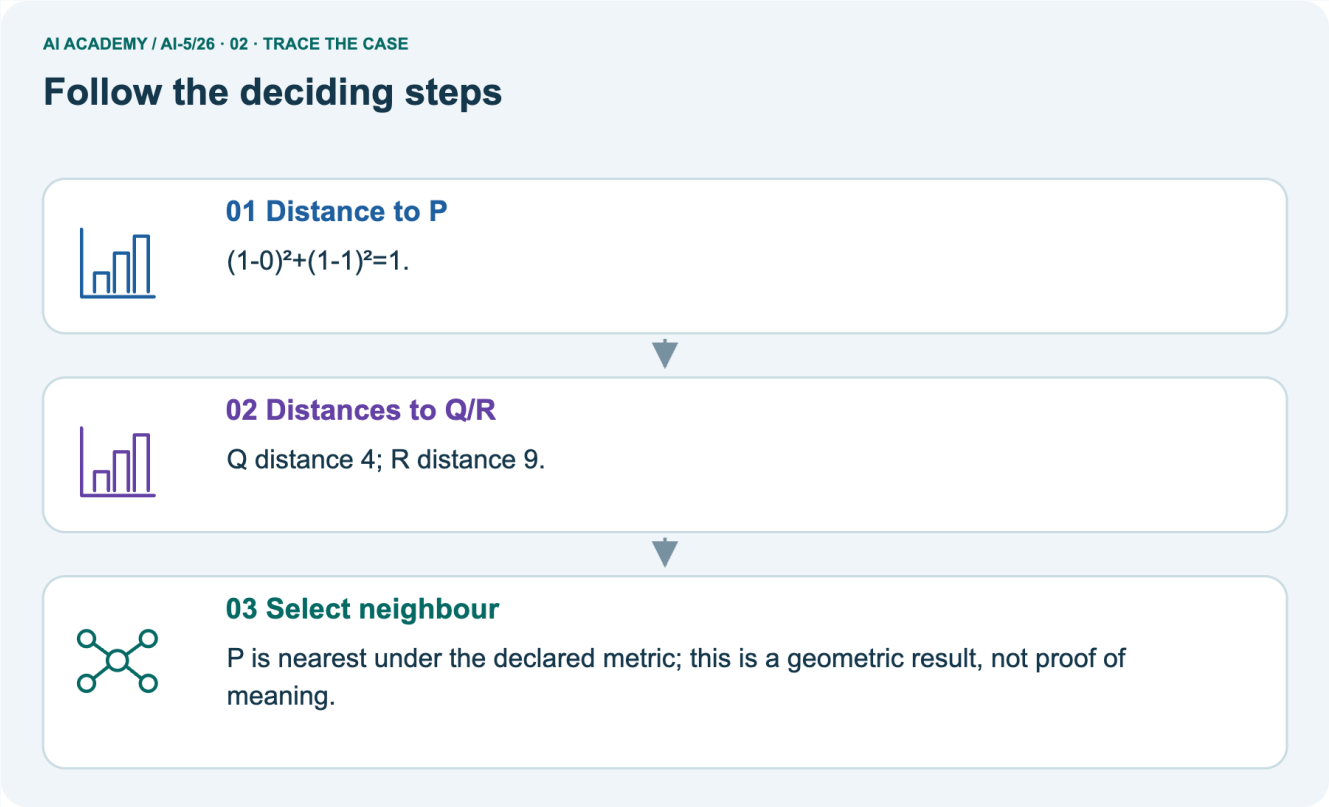
Follow the labelled picture with the matching explanation. Open the vector diagram

Measure embedding neighbours without turning numerical proximity into factual certainty. Supplied facts: Toy table P =[0, 1], Q=[3, 1], R=[1, 4]; query [1, 1]. Use **squared Euclidean distance**.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

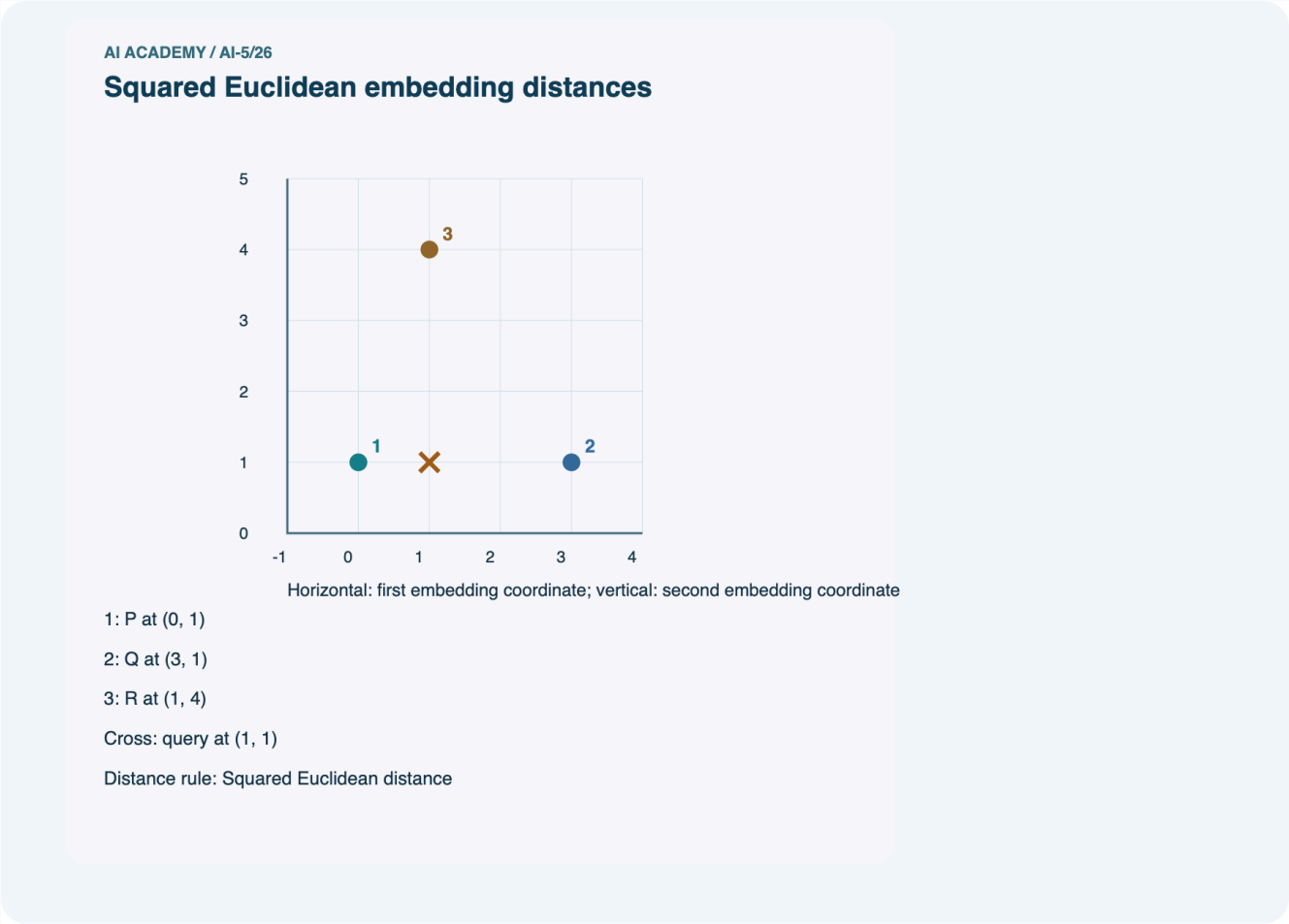


Follow the labelled picture with the matching explanation. Open the vector diagram

Distance to P: (1-0)²+(1-1)²=1. → Distances to Q/R: Q distance 4; R distance 9. → Select neighbour: P is nearest under the declared metric; this is a geometric result, not proof of meaning.

Point to each numbered stage and explain which recorded **input** it uses.

03. Squared Euclidean embedding distances

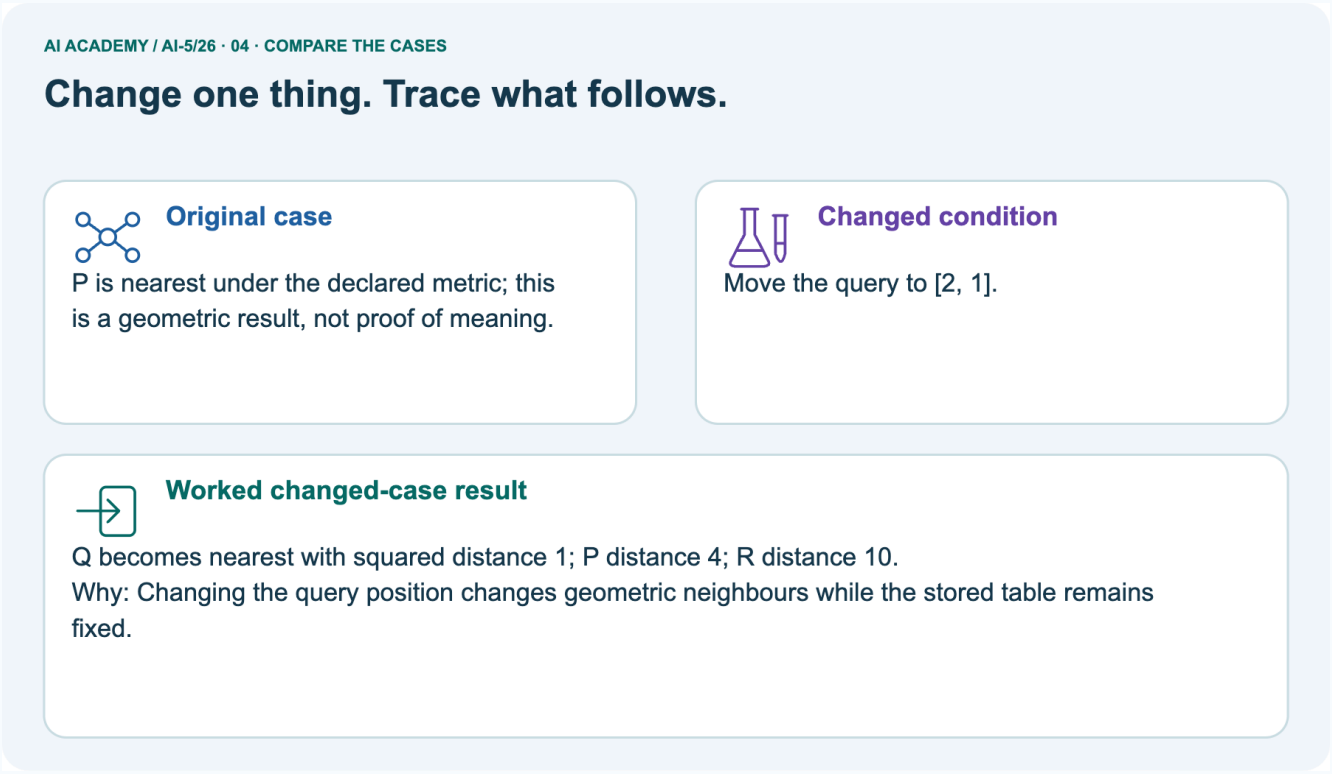


Follow the labelled picture with the matching explanation. Open the vector diagram

Squared Euclidean embedding distances. P: (0, 1). Q: (3, 1). R: (1, 4). Query query: (1, 1). Axes: first embedding coordinate; second embedding coordinate. Distance: **Squared Euclidean distance**.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



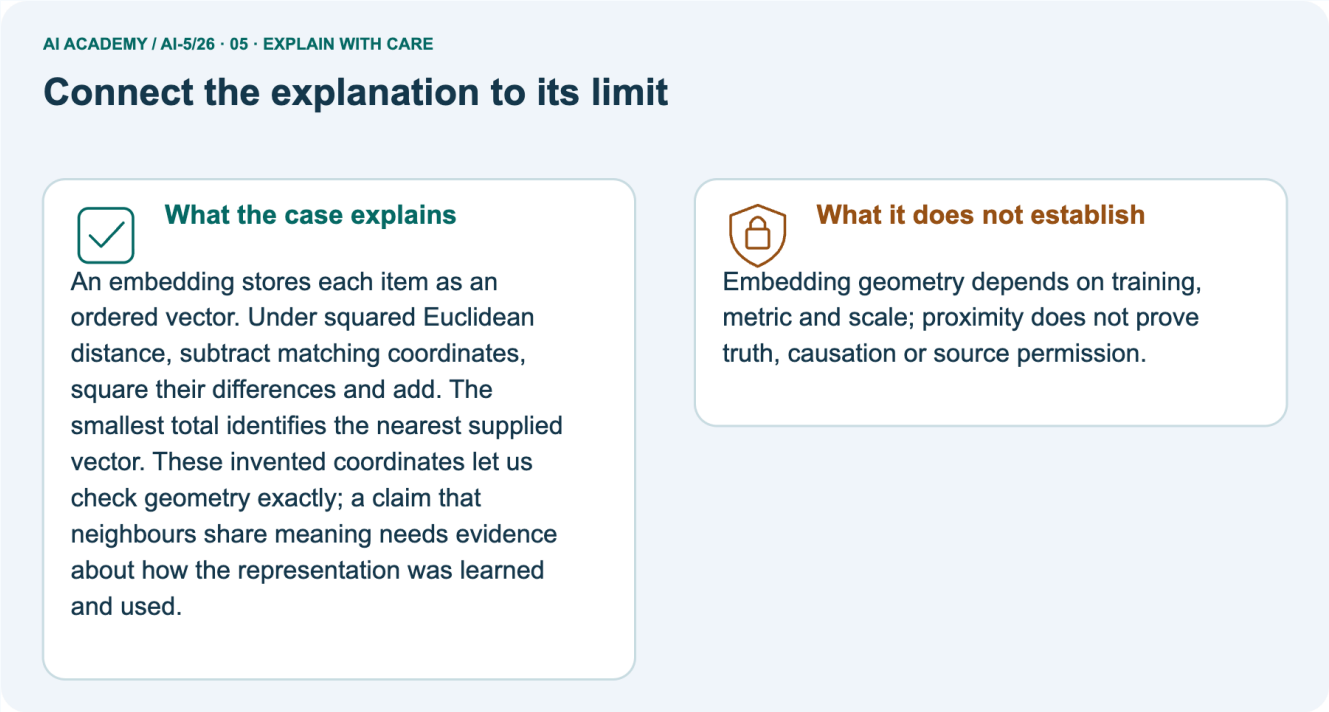
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: P is nearest under the declared metric; this is a geometric result, not proof of meaning. Changed condition: Move the query to [2, 1].

Expected result: Q becomes nearest with squared distance 1; P distance 4; R distance 10. Reason: Changing the query position changes geometric neighbours while the stored table remains fixed.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

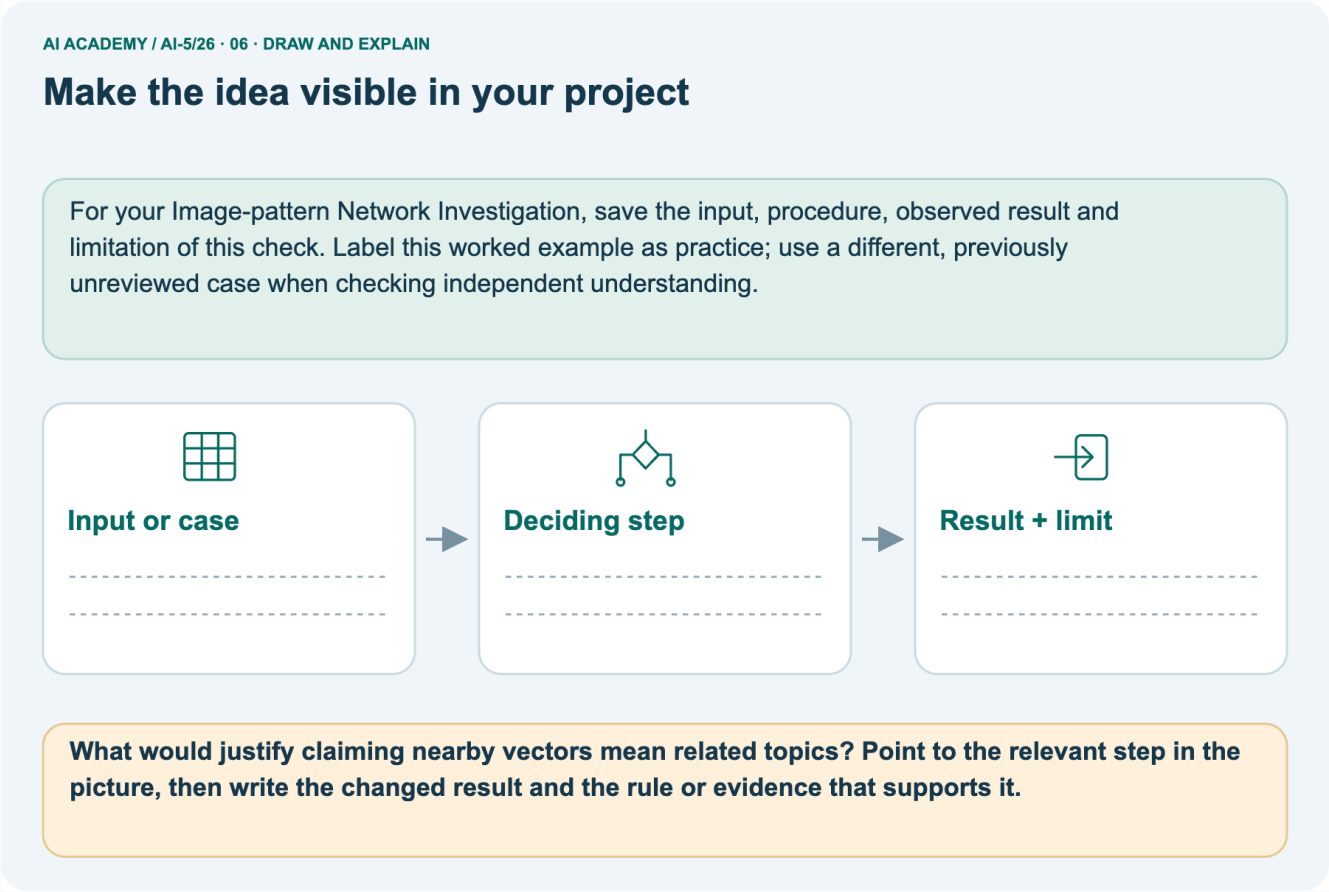
An embedding stores each item as an ordered vector. Under **squared Euclidean distance**, subtract matching coordinates, square their differences and add. The smallest total identifies the nearest supplied vector.

These invented coordinates let us check geometry exactly; a claim that neighbours share meaning needs **evidence** about how the representation was learned and used.

Limit: Embedding geometry depends on **training**, metric and scale; proximity does not prove truth, causation or source permission.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: What would justify claiming nearby vectors mean related topics? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Embeddings and neighbourhoods: the case

02. Follow the deciding steps

03. Squared Euclidean embedding distances

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.